Aim: Write a program to demonstrate creating object a class and call the methods of the class with different access modifiers (public, and private)

Program:

```
class Demo {

   // Public method
   public void publicMethod() {

      System.out.println("This is a PUBLIC method.");
      // Calling private method inside the same class
      privateMethod();
   }

   // Private method
   private void privateMethod() {
      System.out.println("This is a PRIVATE method.");
   }
}

public class Main {
   public static void main(String[] args) {
      // Creating object of Demo class
      Demo obj = new Demo();

      // Calling public method (allowed)
      obj.publicMethod();


      // Calling private method (NOT allowed directly)
      // obj.privateMethod();
   }
}
```

Output:

```
This is a PUBLIC method.
This is a PRIVATE method.
=== Code Execution Successful ===
```

Aim: Write a program to demonstrate garbage collection through the finalize method () with a suitable example.

Program:

```
class GarbageDemo {
   public GarbageDemo() {
      System.out.println("Object created");
   }
}

public class Main {
   public static void main(String[] args) {

      GarbageDemo obj1 = new GarbageDemo();
      GarbageDemo obj2 = new GarbageDemo();

      obj1 = null;
      obj2 = null;

      System.gc();

      System.out.println("End of main method");
   }
}
```

Output:

```
Object created

Object created

End of main method


=== Code Execution Successful ===
```

Aim: Write a program to demonstrate inheritance single and multilevel with a suitable example code.

Program:

```
// Base class
class Animal {
   void eat() {
      System.out.println("Animal is eating");
   }
}

// Single Inheritance: Dog inherits Animal
class Dog extends Animal {
   void bark() {
      System.out.println("Dog is barking");
   }
}

// Multilevel Inheritance: Puppy inherits Dog (which already inherits Animal)
class Puppy extends Dog {
   void weep() {
      System.out.println("Puppy is weeping");
   }
}

public class Main {
   public static void main(String[] args) {

      // Demonstrating Single Inheritance
```

```
        Dog d = new Dog();
        d.eat();   // inherited from Animal
        d.bark();  // Dog's own method

        System.out.println();

        // Demonstrating Multilevel Inheritance
        Puppy p = new Puppy();
        p.eat();   // from Animal
        p.bark();  // from Dog
        p.weep();  // Puppy's own method
    }
}
```

Output:

```
Animal is eating
Dog is barking

Animal is eating
Dog is barking
Puppy is weeping

=== Code Execution Successful ===
```

Aim: Write a program to demonstrate interface and abstract class with a suitable example code.

Program:

```
// Interface
interface Printable {
   void print();
}

// Abstract class
abstract class Shape {
   abstract void draw();   // abstract method

   void message() {       // concrete method
      System.out.println("This is a Shape");
   }
}

// Class implementing interface and extending abstract class
class Circle extends Shape implements Printable {

   @Override
   void draw() {
      System.out.println("Drawing Circle");
   }

   @Override
   public void print() {
      System.out.println("Printing Circle");
```

```
    }
}

public class Main {
    public static void main(String[] args) {

        Circle c = new Circle();

        c.message(); // from abstract class
        c.draw();    // implemented abstract method
        c.print();   // implemented interface method
    }
}
```

Output:

```
This is a Shape
Drawing Circle
Printing Circle

=== Code Execution Successful ===
```

Aim: Write a program to demonstrate how to create a package with a suitable example code.

Program:

```
// File: MyPackage/Demo.java
package MyPackage;

public class Demo {

   public void show() {
      System.out.println("This is a class inside MyPackage.");
   }
}

// File: Main.java
import MyPackage.Demo;

public class Main {
   public static void main(String[] args) {

      // Creating object of package class
      Demo obj = new Demo();

      // Calling method
      obj.show();
   }
}
```

How to Compile and Run:

1. Compile package class:

   javac -d . Demo.java

2. Compile main class:

   javac Main.java

3. Run:

   java Main

Output:

```
This is a class inside MyPackage.
=== Code Execution Successful ===
```

Aim: Write a program to demonstrate handling the String through String class of Java. Use suitable example.

Program:

```
public class Main {
   public static void main(String[] args) {

      // Creating strings
      String str1 = "Hello";
      String str2 = "World";

      // Concatenation
      String result = str1.concat(" ").concat(str2);
      System.out.println("Concatenated String: " + result);

      // Length of string
      System.out.println("Length: " + result.length());

      // Convert to uppercase
      System.out.println("Uppercase: " + result.toUpperCase());

      // Convert to lowercase
      System.out.println("Lowercase: " + result.toLowerCase());

      // Character at specific index
      System.out.println("Character at index 1: " + result.charAt(1));

      // Substring
      System.out.println("Substring (0 to 5): " + result.substring(0, 5));
```

```
        // Comparison
        if (str1.equals("Hello")) {
            System.out.println("Strings are equal");
        }
    }
}
```

Output:

```
Concatenated String: Hello World
Length: 11
Uppercase: HELLO WORLD
Lowercase: hello world
Character at index 1: e
Substring (0 to 5): Hello
Strings are equal

=== Code Execution Successful ===
```

Aim: Write a program to demonstrate to create own exception class by using a suitable example code.

Program:

```
// Custom Exception Class
class InvalidAgeException extends Exception {

   // Constructor
   InvalidAgeException(String message) {
      super(message);
   }
}

public class Main {

   // Method to check age
   static void checkAge(int age) throws InvalidAgeException {
      if (age < 18) {
         // Throw custom exception
         throw new InvalidAgeException("Age is less than 18. Not eligible.");
      } else {
         System.out.println("Eligible to vote.");
      }
   }

   public static void main(String[] args) {

      try {
         checkAge(16);  // Change value to test
```

```
        } catch (InvalidAgeException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}
```

Output:

```
Exception caught: Age is less than 18. Not eligible.

=== Code Execution Successful ===
```

Aim: Write a program to create multi-thread using Runnable interface with a suitable example code.

Program:

```
// Class implementing Runnable interface
class MyThread implements Runnable {

    @Override
    public void run() {
        // Code executed by thread
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread is running: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {

        // Create Runnable object
        MyThread obj = new MyThread();

        // Create Thread object and pass Runnable
        Thread t1 = new Thread(obj);

        // Start thread
        t1.start();

        // Main thread work
```

```
        for (int i = 1; i <= 5; i++) {
            System.out.println("Main thread: " + i);
        }
    }
}
```

Output:

```
Thread is running: 1
Thread is running: 2
Thread is running: 3
Thread is running: 4
Thread is running: 5
Main thread: 1
Main thread: 2
Main thread: 3
Main thread: 4
Main thread: 5

=== Code Execution Successful ===
```

Aim: Write a program to demonstrate isAlive() method of the thread class with a suitable example code

Program:

```
// Class extending Thread
class MyThread extends Thread {

   @Override
   public void run() {
      System.out.println("Thread is running...");
      try {
         Thread.sleep(1000); // pause for 1 second
      } catch (InterruptedException e) {
         System.out.println(e);
      }
      System.out.println("Thread finished execution.");
   }
}

public class Main {
   public static void main(String[] args) {

      MyThread t1 = new MyThread();

      // Before starting thread
      System.out.println("Is thread alive? " + t1.isAlive());

      t1.start();
```

```
        // After starting thread
        System.out.println("Is thread alive? " + t1.isAlive());

        try {
            t1.join(); // wait for thread to finish
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        // After thread execution
        System.out.println("Is thread alive? " + t1.isAlive());
    }
}
```

Output:

```
Is thread alive? false
Thread is running...
Is thread alive? true
Thread finished execution.
Is thread alive? false

=== Code Execution Successful ===
```

Aim: write a program to demonsrate read and write file contet from and to the file using. FileInputStream and FileOutputStream class.

Program:

```
import java.io.FileInputStream;
        import java.io.FileOutputStream;
        import java.io.IOException;

        public class Main {
           public static void main(String[] args) {

              String data = "Hello, this is file handling in Java.";

              // Writing to file
              try {
                 FileOutputStream fos = new FileOutputStream("sample.txt");
                 fos.write(data.getBytes()); // convert string to bytes
                 fos.close();
                 System.out.println("Data written to file successfully.");
              } catch (IOException e) {
                 System.out.println("Error in writing file: " + e);
              }

              // Reading from file
              try {
                 FileInputStream fis = new FileInputStream("sample.txt");
                 int i;

                 System.out.println("Reading data from file:");
                 while ((i = fis.read()) != -1) {
```

```
                System.out.print((char) i);
            }

            fis.close();
        } catch (IOException e) {
            System.out.println("Error in reading file: " + e);
        }
    }
}
```

Output:

```
Data written to file successfully.
Reading data from file:
Hello, this is file handling in Java.
=== Code Execution Successful ===
```

Aim: write a program to demonstrate how to send and recive text message using TCP/IP socket b/w lient and server.

Program :

server.java

```
import java.io.*;
import java.net.*;

class Server {
   public static void main(String[] args) {
      try {
         // Create server socket on port 5000
         ServerSocket server = new ServerSocket(5000);
         System.out.println("Server is waiting for client...");

         // Accept client connection
         Socket socket = server.accept();
         System.out.println("Client connected.");

         // Input and Output streams
         BufferedReader in = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));
         PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

         // Receive message from client
         String clientMsg = in.readLine();
         System.out.println("Client says: " + clientMsg);
```

```
            // Send response to client
            out.println("Hello from Server");

            // Close resources
            socket.close();
            server.close();

        } catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

Client Program

```
import java.io.*;
import java.net.*;

class Client {
    public static void main(String[] args) {
        try {
            // Connect to server on localhost port 5000
            Socket socket = new Socket("localhost", 5000);
            System.out.println("Connected to server.");

            // Input and Output streams
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
```

```
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

            // Send message to server
            out.println("Hello from Client");

            // Receive response from server
            String serverMsg = in.readLine();
            System.out.println("Server says: " + serverMsg);

            // Close resources
            socket.close();

        } catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

Server Output:

```
Server is waiting for client...
Client connected.
Client says: Hello from Client
```

Client Output:

```
Connected to server.
```

Server says: Hello from Server

Aim: Write a program to create an I/O interface by using necessary suing components

Program:

```
import javax.swing.*;
import java.awt.event.*;

public class IOInterface {

   public static void main(String[] args) {

      // Create frame
      JFrame frame = new JFrame("I/O Interface");
      frame.setSize(400, 200);
      frame.setLayout(null);

      // Label
      JLabel label = new JLabel("Enter your name:");
      label.setBounds(30, 30, 150, 30);
      frame.add(label);

      // Text field (Input)
      JTextField textField = new JTextField();
      textField.setBounds(180, 30, 150, 30);
      frame.add(textField);

      // Button
```

```
        JButton button = new JButton("Submit");
        button.setBounds(130, 80, 100, 30);
        frame.add(button);

        // Label for output
        JLabel result = new JLabel("");
        result.setBounds(30, 120, 300, 30);
        frame.add(result);

        // Button action (I/O handling)
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String name = textField.getText(); // Input
                result.setText("Hello, " + name);  // Output
            }
        });

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Output:

Aim: write a program to handle even of all source of event source component of java swing.

Program:

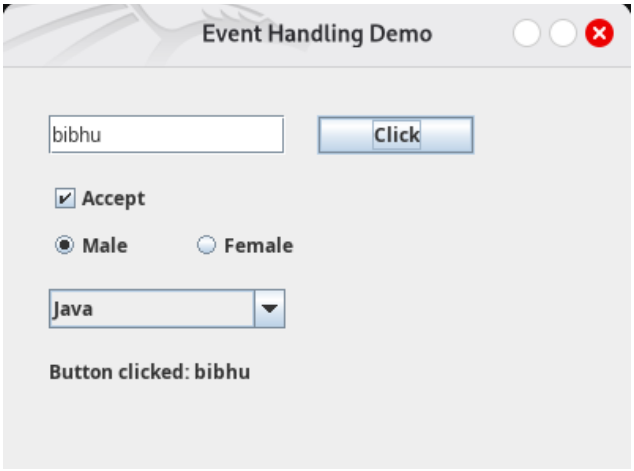
```
import javax.swing.*;
        import java.awt.event.*;


        public class EventDemo extends JFrame implements ActionListener, ItemListener
        {


           JTextField textField;
           JButton button;
           JCheckBox checkBox;
           JRadioButton r1, r2;
           JComboBox<String> comboBox;
           JLabel result;


           public EventDemo() {
              setTitle("Event Handling Demo");
              setSize(400, 300);
              setLayout(null);


              // TextField
              textField = new JTextField();
              textField.setBounds(30, 30, 150, 25);
              add(textField);


              // Button
              button = new JButton("Click");
              button.setBounds(200, 30, 100, 25);
```

```
add(button);

// CheckBox
checkBox = new JCheckBox("Accept");
checkBox.setBounds(30, 70, 100, 25);
add(checkBox);

// Radio Buttons
r1 = new JRadioButton("Male");
r2 = new JRadioButton("Female");
r1.setBounds(30, 100, 80, 25);
r2.setBounds(120, 100, 80, 25);

ButtonGroup bg = new ButtonGroup();
bg.add(r1);
bg.add(r2);

add(r1);
add(r2);

// ComboBox
String[] items = {"Java", "Python", "C++"};
comboBox = new JComboBox<>(items);
comboBox.setBounds(30, 140, 150, 25);
add(comboBox);

// Result Label
result = new JLabel("");
```

```
        result.setBounds(30, 180, 300, 25);
        add(result);

        // Register listeners
        button.addActionListener(this);
        textField.addActionListener(this);
        checkBox.addItemListener(this);
        r1.addItemListener(this);
        r2.addItemListener(this);
        comboBox.addActionListener(this);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }
    // Handle Action Events
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == button) {
            result.setText("Button clicked: " + textField.getText());
        } else if (e.getSource() == textField) {
            result.setText("Text entered: " + textField.getText());
        } else if (e.getSource() == comboBox) {
            result.setText("Selected: " + comboBox.getSelectedItem());
        }
    }

    // Handle Item Events
    public void itemStateChanged(ItemEvent e) {
        if (e.getSource() == checkBox) {
```

```
            result.setText("Checkbox: " + (checkBox.isSelected() ? "Checked" :
"Unchecked"));
        } else if (e.getSource() == r1 || e.getSource() == r2) {
            result.setText("Gender: " + (r1.isSelected() ? "Male" : "Female"));
        }
    }

    public static void main(String[] args) {
        new EventDemo();
    }
}
```

Output:



Aim: create an I/O interface with database connectivity and insert and retrive data in the table.

Program:

Database Table (MySQL)

```
CREATE DATABASE testdb;

USE testdb;

CREATE TABLE student (
   id INT PRIMARY KEY,
   name VARCHAR(50)
);
```

Java Program (Swing + JDBC)

```
import javax.swing.*;
import java.awt.event.*;
import java.sql.*;

public class DBInterface extends JFrame implements ActionListener {

   JTextField idField, nameField;
   JButton insertBtn, viewBtn;
   JTextArea area;

   // Database connection
   Connection con;

   public DBInterface() {
```

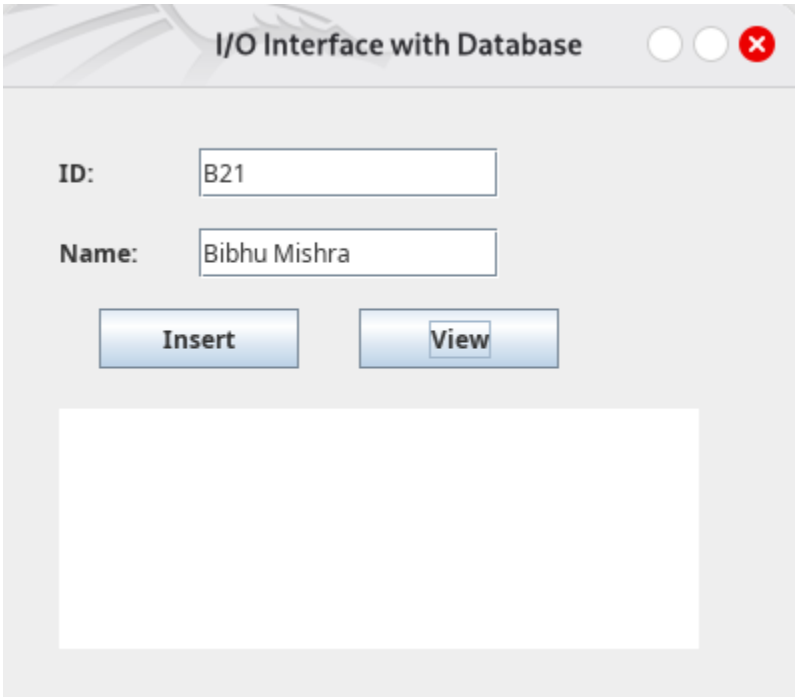
```
setTitle("I/O Interface with Database");
setSize(400, 350);
setLayout(null);

JLabel l1 = new JLabel("ID:");
l1.setBounds(30, 30, 50, 25);
add(l1);

idField = new JTextField();
idField.setBounds(100, 30, 150, 25);
add(idField);

JLabel l2 = new JLabel("Name:");
l2.setBounds(30, 70, 50, 25);
add(l2);

nameField = new JTextField();
nameField.setBounds(100, 70, 150, 25);
add(nameField);

insertBtn = new JButton("Insert");
insertBtn.setBounds(50, 110, 100, 30);
add(insertBtn);

viewBtn = new JButton("View");
viewBtn.setBounds(180, 110, 100, 30);
add(viewBtn);
```

```
        area = new JTextArea();
        area.setBounds(30, 160, 320, 120);
        add(area);

        insertBtn.addActionListener(this);
        viewBtn.addActionListener(this);

        // Connect to database
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password"
            );
        } catch (Exception e) {
            System.out.println(e);
        }

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {

        try {
            if (e.getSource() == insertBtn) {

                String id = idField.getText();
```

```
            String name = nameField.getText();

            PreparedStatement ps = con.prepareStatement(
                  "INSERT INTO student VALUES (?, ?)");
            ps.setInt(1, Integer.parseInt(id));
            ps.setString(2, name);

            ps.executeUpdate();
            area.setText("Data Inserted Successfully");

         } else if (e.getSource() == viewBtn) {

            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM student");

            area.setText("");
            while (rs.next()) {
               area.append(rs.getInt(1) + "  " + rs.getString(2) + "\n");
            }
         }
      } catch (Exception ex) {
         System.out.println(ex);
      }
   }

   public static void main(String[] args) {
      new DBInterface();
   }
```

```
}
```

Output:


I/O Interface with Database
ID:
B21
Name:
Bibhu Mishra
Insert
View

Aim: write a program to demomnstrate how to pass paramere from HTML documents to applet with a sutaible code example

Program:

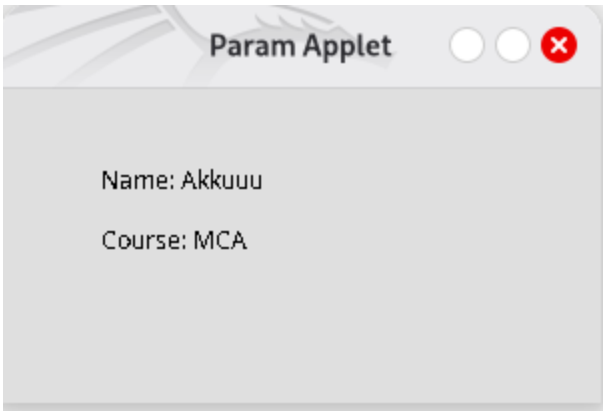
Applet Program

```
import java.applet.Applet;
import java.awt.Graphics;
import java.awt.Frame;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

public class ParamApplet extends Applet {

String name;
String course;

public void init() {
// Get parameters from HTML
try {
name = getParameter("username");
course = getParameter("course");
} catch (NullPointerException e) {
// Ignored when running as standalone app
}
}

public void paint(Graphics g) {
g.drawString("Name: " + name, 50, 50);
g.drawString("Course: " + course, 50, 80);
}

public static void main(String[] args) {
Frame frame = new Frame("Param Applet");
ParamApplet applet = new ParamApplet();

String html = "";
try {
html = new
String(java.nio.file.Files.readAllBytes(java.nio.file.Paths.get("ParamApplet.html"
)));
} catch (Exception e) {
System.err.println("Could not read ParamApplet.html");
}

applet.name = extractParam(html, "username", "Unknown");
applet.course = extractParam(html, "course", "Unknown");

frame.add(applet);
frame.setSize(300, 200);
```

```
frame.addWindowListener(new WindowAdapter() {
public void windowClosing(WindowEvent we) {
System.exit(0);
}
});

applet.init();
applet.start();
frame.setVisible(true);
}

private static String extractParam(String html, String paramName, String
defaultVal) {
java.util.regex.Matcher m = java.util.regex.Pattern
.compile("(?i)<param\\s+name=[\"']?" + paramName +
"[\"']?\\s+value=[\"']?([^\"'>]+)[\"']?\\s*>")
.matcher(html);
if (m.find()) {
return m.group(1);
}
return defaultVal;
}
}
```

HTML FILE:

```
<html>

<body>
  <applet code="ParamApplet.class" width="300" height="200">
    <param name="username" value="Akkuuu">
    <param name="course" value="MCA">
  </applet>
</body>

</html>
```


Param Applet
Name: Akkuuu
Course: MCA